



Adaptive Path Planning and Collision Avoidance for Autonomous Vehicles on Unstructured Indian Roads

Problem statement ID: 26037

Team name- Epsilon



Problem statement

Design and simulate an adaptive path planning system for an autonomous vehicle that operates in unstructured Indian road conditions. The system should perceive the environment using a multi-sensor setup such as camera, LiDAR, and radar, and identify diverse road users and obstacles, including auto-rickshaws, pushcarts, pedestrians, and animals. It should predict the short-term motion of surrounding agents, including non lane-based and irregular movement patterns, and generate a safe, collision-free path that can be replanned in real time. The solution should also handle practical driving situations such as missing lane markings, informal merging, sudden pedestrian movement, and unexpected obstacles on the road. Teams should validate their solution using at least five realistic Indian road scenarios: an unmarked village road, a busy urban intersection without signals, a highway merge involving slow-moving vehicles, a dense market area with mixed traffic, and a sudden cattle-crossing event. Teams are encouraged to use MathWorks tools such as RoadRunner for scenario design, Automated Driving Toolbox for sensor modeling and fusion, Navigation Toolbox and Stateflow for planning and decision logic, Vehicle Dynamics Blockset or a Simulink bicycle model for vehicle behavior, and Deep Learning Toolbox for detection and trajectory prediction.



POC Overview

Perception — Detect and understand vehicles, pedestrians, animals & road hazards

Tracking & Prediction — Track surrounding agents and predict their non-lane-based motion

Sensor Fusion — Combine Camera, LiDAR & Radar for robust environment awareness

Costmap & Risk Modeling — Represent dynamic agents, potholes and uncertainty as planning costs

Path Planning & Decision Logic — Generate safe, collision-free paths using **Stateflow + Hybrid A***

Vehicle Dynamics & Control — Convert planned paths into smooth steering, throttle & braking commands

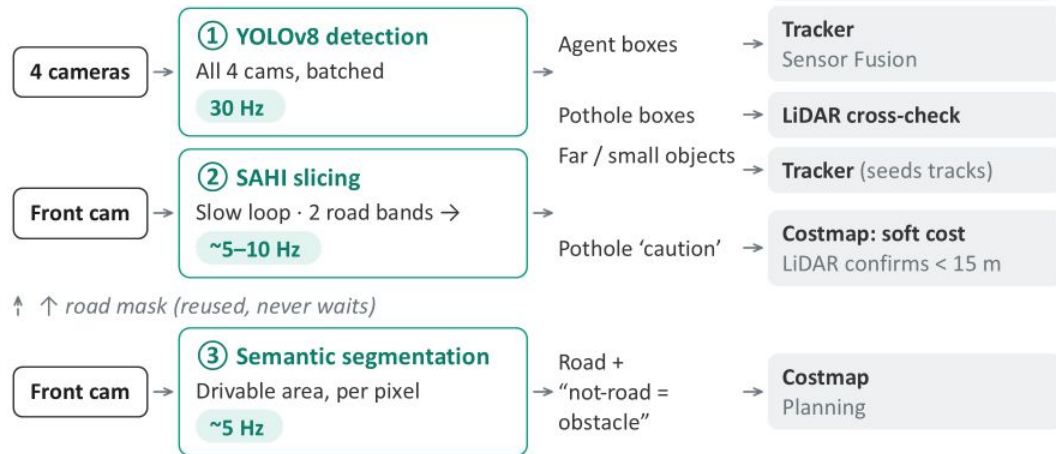
Scenario Validation — Test the complete pipeline across diverse **Indian road scenarios**



Perception: What the Car Sees, and How

Things get boxes. Stuff gets pixels. Small, far things get a closer look.

CAMERA PIPELINE



Runs entirely on PS toolboxes: Computer Vision · Deep Learning · Automated Driving

Simulation 3D Camera yolov8ObjectDetector selectStrongestBboxMulticlass
semanticseg (DeepLabv3+) vehicleCostmap Rate Transition (multi-rate)

Three tools, each doing only the job it's best at, all inside Simulink.

WHY THIS DESIGN

YOLOv8 for agents

Best of 5 models on the Indian DriveIndia benchmark: **78.7% mAP50** [1]. MathWorks package supports Simulink + custom training [3].

Semantic seg for road

Road and mud are "stuff". Only per-pixel labels can map them [4].

YOLOv8-seg: not used

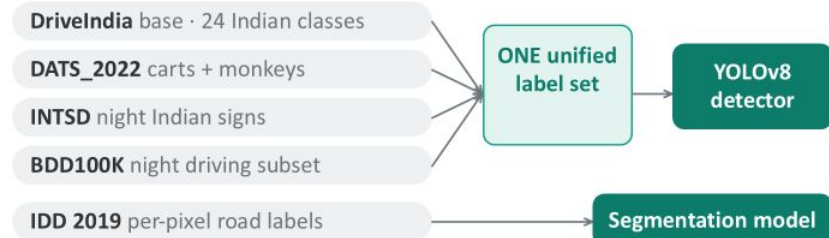
No official Simulink or MATLAB training support [3], and no instance masks for most Indian classes.

SAHI: where it looks

Far band 40–150 m + pothole band 15–30 m. Rows from camera geometry + car tilt, refined by the road mask.

Training Data: Built for Indian Roads

FOUR SETS IN, ONE DETECTOR OUT



Merged at training time → one model, one pass, zero extra runtime cost

A flat pothole vanishes with distance · image height (px, log)



At 80 km/h: ~0.3–0.7 s warning vs 1.2 s needed.

→ Pothole avoidance is a low-speed capability.

SAHI 'caution' (15–30 m) slows early; LiDAR measures < 15 m.

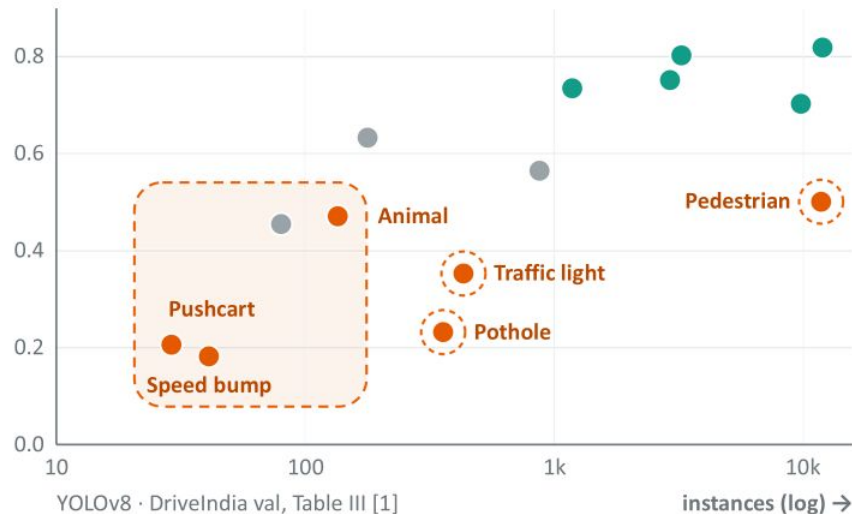
illustrative: cam 1.5 m high, 1280 px / 60° FOV, pothole 0.6 m, car 1.5 m

Where the detector struggles, and why

Too little data → dataset merge

Small or hidden → SAHI for small · crowd NMS + safety nets for hidden

mAP50:95 ↑



YOLOv8 · DriveIndia val, Table III [1]

instances (log) →

Weak classes are a data problem or a size problem. We attack each with a different fix.

One Pipeline, Five Scenarios: Adaptive Perception

	ON / heavy low rate off limitation				
Scenario →	Village road	Urban intersection	Highway merge	Dense market	Cattle crossing
Main workhorse	Segmentation	Detection + tracker	Radar + long range	Seg + LiDAR safety nets	LiDAR “something’s there”
Cameras	Front	Front + sides	Front + rear + right	All 4	Front + sides
SAHI	Signage + pothole caution	Off	Slow loop, horizon	Off (crowding ≠ small)	Off
Potholes	Active	Off	⚠ Limitation	Active	—
Weakest link	Animals	Pedestrians	Pothole lead time	Unknown objects	Occluded entry

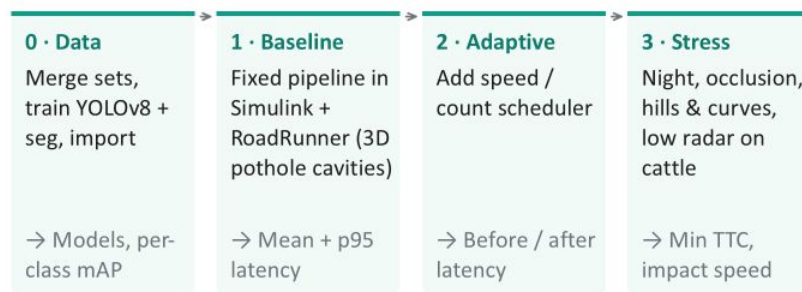
HOW IT SWITCHES



No scenario labels needed: it switches on signals the car already knows.

Slow jobs staggered across frames → flat p95 latency (PS scores mean + p95).

ROADMAP



STATED LIMITATIONS

Potholes: low speed only (≤ 45 km/h, set by 15 m LiDAR range)
Animals stepping out inside stopping distance → planner slows near blind zones
No species-level animal class
Dense crowds tracked as zones, not individuals

Different roads need different perception. Ours adapts automatically, and every block runs in MATLAB/Simulink.

The Trajectory Prediction Crisis & Non-Lane Motion Modeling



Objective: Replace deterministic, lane-anchored motion models with multi-modal spatial probability networks capable of anticipating the opportunistic gap-filling and hazard-induced swerving inherent to mixed traffic.

- **The Failure of Lane-Centric Kinematics:** Standard trajectory prediction models act like trains on invisible tracks, anchoring their math to lane centerlines. Because formal lane discipline is virtually non-existent in Indian traffic, these algorithms fail catastrophically. The system must predict motion in an unconstrained, two-dimensional coordinate space where any physical tarmac is a valid path.
- **Micro-Gap Filling & High-Frequency Lateral Dynamics:** Indian traffic features opportunistic "gap-filling," where two-wheelers and auto-rickshaws execute high-frequency lateral swerves to exploit micro-gaps in moving traffic. These agents exhibit extreme non-holonomic agility that breaks standard Kalman filter prediction envelopes, requiring sequence models trained explicitly on real-world Indian swerving behaviors.
- **Causal Opacity & Hazard-Induced Swerving:** To a standard AI, a vehicle violently swerving on an empty road looks like unlearnable noise. The hidden cause is often a deep pothole or an unpaved shoulder. If the perception system does not explicitly feed these "static hazards" into the prediction network, the AI cannot learn the causal relationship. The architecture must force cross-attention between the moving agents (Track Queries) and road defects (Map Queries) to accurately predict evasive maneuvers.
- **Multi-Modal Uncertainty (The "Freezing Robot" Antidote):** Indian traffic relies on informal negotiation (e.g., nudging forward at a junction to force right-of-way) rather than rigid traffic signals, making the exact future of an agent mathematically unknowable. If an AI attempts to predict a single deterministic line, it will frequently be wrong and abruptly brake, suffering from "Freezing Robot Syndrome."

Proposed Solution: Prediction/Trajectory

Objective: Predict the multi-modal, non-lane-based future paths of surrounding agents using self-attention mechanisms in 3D space.

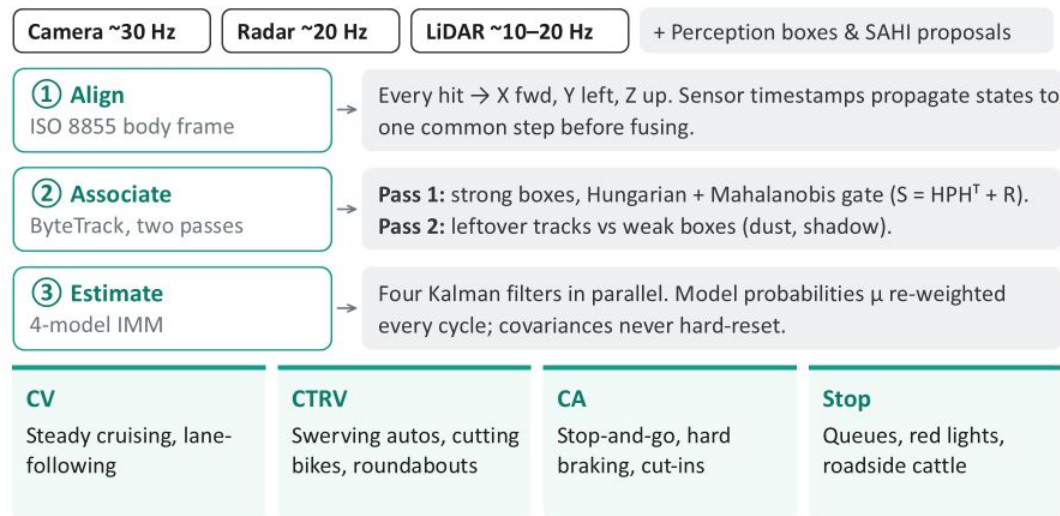
- **Attention in BEV Space (The MotionFormer):** Emulating the UniAD architecture, this pipeline utilizes a "MotionFormer"—a specialized Transformer module that acts as an interaction engine by launching learnable agent queries to cross-reference other moving vehicles and the static road topology. By executing this cross-attention directly in the 3D BEV coordinate space (via MATLAB's `selfAttentionLayer`), the network evaluates the spatial relationship between the IMM Track Queries (vehicles) and the 3D Map Queries (potholes and road boundaries).
- **Causal Hazard Avoidance:** The Transformer is fine-tuned exclusively on the Mendeley Chennai mixed-traffic trajectory dataset. By forcing the network to attend to the static Pothole Tokens, it mathematically learns the causal dynamics of unstructured roads: *If a two-wheeler approaches a negative-Z pothole, the probability of a lateral X-Y swerve increases.*
- **Multi-Modal Future Generation (GMM):** Because Indian traffic is highly non-deterministic, the prediction module outputs a Gaussian Mixture Model (GMM) representing spatial probability rather than a single fixed path.
 - **The Math:** For each agent, it predicts K possible future trajectories over a set time horizon (e.g., t+3seconds). Each trajectory mode outputs a sequence of (x,y) coordinates, a spatial covariance matrix (representing a physical uncertainty ellipse around the vehicle at each timestep), and an overall probability weight summing to 1.
 - **Costmap Integration:** These GMM spatial probability ellipses are projected directly into the downstream planner's `vehicleCostmap`. This allows the ego-vehicle to mathematically calculate exactly how much "safe clearance" it needs to leave around an erratic auto-rickshaw based on the neural network's uncertainty bounds.
- **Core Literature:**
 - *Hu et al. (2023).* "Planning-oriented Autonomous Driving" (UniAD).
 - *Canagaraj et al. (2015).* "Trajectory Data and Flow Characteristics of Mixed Traffic".

Sensor Fusion: Multi-Modal Tracking & Occlusion Resilience

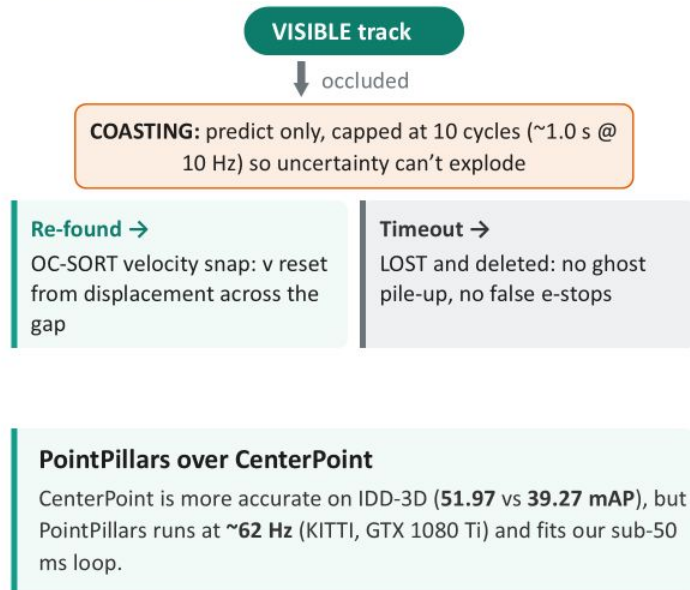
One frame, one clock, four motion guesses. Hidden agents keep moving on the map.

FUSION PIPELINE

rates: nominal, to be benchmarked



OCCUSION HANDLING



Four motion models, two matching passes, one bounded coast: tracks survive dense Indian traffic without false stops.

Sensor Fusion: 3D Pothole Quantification & Costmap Hand-Off

The camera says “maybe” from afar. LiDAR measures it up close. People always outrank potholes.

DUAL-RATE LOOKAHEAD

low-speed capability

0–15 m · LiDAR · 10–20 Hz

Measures depth and volume

15–30 m · Camera · ~5–10 Hz

Pothole boxes + SAHI: “caution” only

Why split: on IDD-3D, LiDAR pedestrian AP falls to **4.48 beyond 25 m** (sparse, occluded returns).

NEAR-FIELD LIDAR PIPELINE · NO TRAINING

① PatchWork++

Polar ring planes fit the asphalt across slopes

② Curvature

$C = \lambda_{\min} / \Sigma \lambda > 0.02$ and > 3 cm below plane

③ Voxelize

$V = N_{\text{vox}} \cdot d^3$, depth in mm

Costmap hand-off	Cost	Planner action
Dip < 5 cm	0.1–0.4 soft	Stateflow SLOW_DOWN, keep lane
Cavity > 5 cm	0.8–1.0 wall	Hybrid A* steers around it
VRU nearby	overrides	VRU boundary wins: slow, drive through

PROOF ON SPARSE LIDAR

100%

detected at 205 pts/m²

0.6 mm

depth error (p = 0.8563)

Point density (pts/m²) → detection %



FALLBACKS

Lead car bounce (ΔZ , pitch rate ω_y) or swerve → flag hidden holes

IMU + wheel odometry bridge 2–5 s GNSS dropouts

Potholes become graded costs, not guesses: measured in 3D, sized in mm, always ranked below human safety.

Path Planning: Challenges and Our Approach

What makes Indian roads hard to plan on, and how we handle each

1. No Lane Markings

THE PROBLEM

Hybrid A* and Frenet planners anchor to a lane centreline. Where markings are absent, worn or ignored, there is no reference frame to plan against.

OUR ANSWER

Plan on free space, not lanes. Fused tracks feed a dynamic occupancy costmap, then Hybrid A*. No lane-detection step in between.

Virtual lane as a soft bias. A cheaper-cost corridor keeps behaviour human-like, yet the vehicle can leave it when a cow or cart blocks it.

Graded costs, not free or blocked. A shallow pothole is crossable at low speed, so it gets a soft cost instead of a wall.

Global route from RoadRunner waypoints stands in for GPS; the local planner handles everything else.

2. Sub-Second Reaction Budget

THE PROBLEM

The PS scores replanning latency. A cattle crossing leaves roughly one second, and a full Hybrid A* search over a fine costmap can exceed it.

OUR ANSWER

Split the work. Stateflow picks the mode, Hybrid A* searches a bounded local window, and a fast MPC layer reacts every step between searches.

Tune grid resolution to the gap that matters, a cart beside a parked auto-rickshaw, then benchmark search time on target hardware.

Budget latency per stage (detect, track, predict, plan, control) and log mean, p95 and worst case in Simulink.

Smooth every replan before the controller sees it, so a sudden new path never becomes a steering spike.

3. Junctions Without Signals

THE PROBLEM

Unsignalled junctions are negotiated by proximity and nudging, and clutter gives low-confidence tracks. Treat all of it as worst case and the vehicle freezes.

OUR ANSWER

Decide on measured quantities: time-to-collision, closing speed and planner feasibility, not traffic rules.

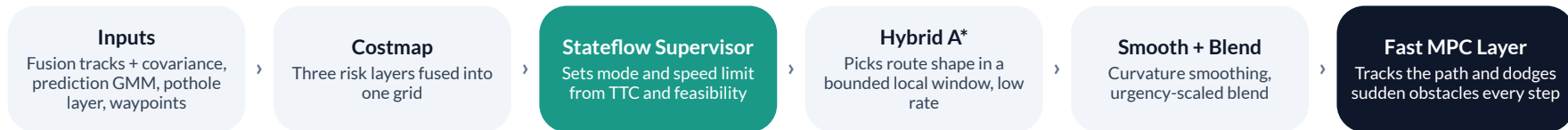
Custom YIELD / NEGOTIATE state for unsignalled junctions, creeping in on proximity and closing speed. Original design; no published template exists.

Graded uncertainty. Covariance and the occlusion flag from fusion scale the cost inflation, instead of one worst-case bubble.

Recovery out of STOP: once the hazard clears the vehicle resumes CRUISE, never a permanent freeze.

Proposed Solution: Stateflow, Hybrid A* and MPC

Stateflow decides the mode, Hybrid A* picks the route shape, MPC reacts every control step



Feedback: if the MPC layer cannot cope, Stateflow triggers a fresh Hybrid A* search; a planner-infeasible flag leads to REROUTE.

1. Decision Logic (Stateflow)

Default is CRUISE. The mode changes on:

SLOW_DOWN	Predicted agent path crosses the ego path; TTC under the soft limit
YIELD / NEGOTIATE	Unsignalled junction; decided by proximity and closing speed
STOP	TTC under the hard limit, e.g. cattle or pedestrian crossing
REROUTE	Planner-infeasible flag; no collision-free path on the costmap
back to CRUISE	Hazard cleared. A STOP held past a timeout escalates to REROUTE

2. Costmap Design

Predicted agent occupancy

GMM uncertainty ellipses from the predictor, projected 2 s ahead in 0.1 s steps. Paths are validated against future occupancy, not a snapshot.

Surface hazards

Shallow pothole (<5 cm): soft cost 0.1 to 0.4, slow and hold line. Deep (>5 cm): 0.8 to 1.0, avoid if space allows, otherwise slow and cross.

Perception uncertainty

Covariance and the VISIBLE / COASTING / LOST flag scale the inflation. Low-confidence clusters are inflated as a whole, not object by object.

3. Metrics and Interfaces

Scored metrics

Replanning latency: timestamps at input and output of each layer; mean, p95 and worst case (cattle crossing).

Path smoothness: peak and mean curvature and lateral jerk, per scenario.

Completion rate: successful runs over N randomised seeds, all five scenarios.

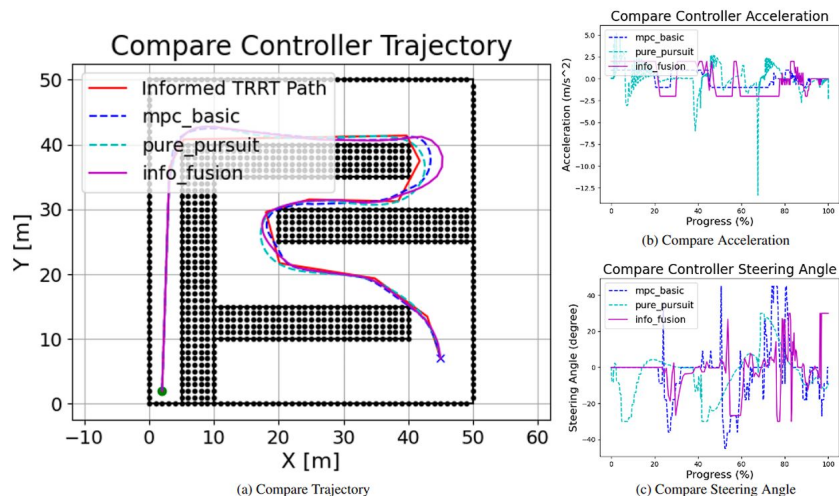
Interfaces with other tracks

Fusion to costmap: pothole cost band, covariance, occlusion flag.

Prediction to Stateflow: minimum TTC and GMM ellipses.

Planner to control: smoothed path and blend window (short for STOP, longer for REROUTE).

Vehicle Dynamics/control



Brief:

A planner can give a perfect path, but if the car cannot follow it smoothly, none of it matters. This track covers how the vehicle turns a planned path into real steering, throttle and brake commands, and how it stays stable even when that path changes suddenly.

Challenges:

- Sudden replans, like a cattle crossing or emergency stop, can jerk the steering and brakes instantly. This happens way too often in indian roads.
- Instant jerks feel unsafe and uncomfortable for passengers
- Path smoothness and completion rate are both scored by the PS, and both depend on how well the controller executes the plan
- A good plan can still fail in execution if the controller cannot track it accurately

Reference Image: Pure Pursuit stays flat for most of the run but has one sharp acceleration spike entering a tight turn. MPC avoids that single spike but oscillates continuously in both acceleration and steering throughout the run, meaning frequent, unpredictable jerks rather than one bad moment.

Proposed Approach: Adaptive Speed-Based Control

Plan:

- Pure Pursuit as default controller for slow, cluttered conditions, village roads, market areas, tight turns
- MPC takes over at higher speed, mainly highway merge, where Pure Pursuit is known to get unstable

Solution:

- Switch decided by vehicle speed, not by scenario, so it adapts automatically as conditions change
- A short blend added at the switch point so the handover itself does not cause a jerk
- Pure Pursuit upgraded to A-PP, lookahead distance adjusts on its own based on speed, curvature and lateral error, instead of staying fixed
- A-PP cuts lateral error notably over standard Pure Pursuit, with no added computation cost (as noted in reference images)

Table 3. Experiment error comparison table.

Method	Average Value of Lateral Error (m)	Variance (m)	Maximum Lateral Error (m)
Pure pursuit	0.02372	0.016588	0.036705
MPC	0.02069	0.014416	0.031628
A-PP	0.01070	0.006663	0.019443



THANK YOU!!